

EPIC 05 — Task 04: Kafka / Event-Driven Communication

The task says:

Create: Order Service → Kafka → Notification Service
Implement event publishing and consumption.

We will create a simple **Order Created** event.

Step 1 — Add Kafka dependency

Add this dependency to **Order Service pom.xml**:

```
<dependency>

    <groupId>org.springframework.kafka</groupId>

    <artifactId>spring-kafka</artifactId>

</dependency>
```

Add the same dependency to **Notification Service pom.xml**.

Step 2 — Start Kafka

If you already have Kafka running from your previous setup, use it.

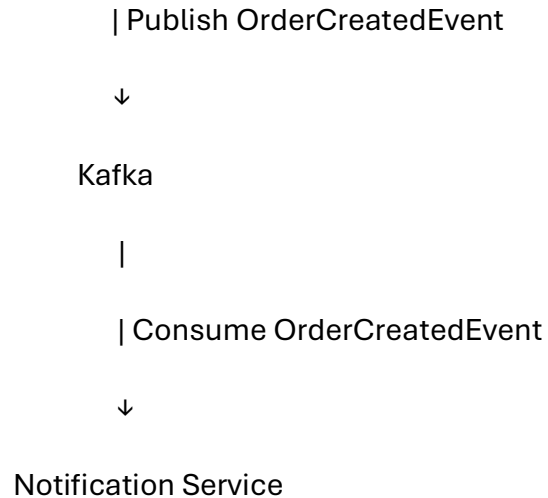
For a simple local setup, Kafka should be available at:

localhost:9092

The architecture will be:

Order Service

|



Step 3 — Configure Order Service

Open:

Order Service

└─ src/main/resources/application.properties

Add:

`spring.kafka.bootstrap-servers=localhost:9092`

`spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.StringSerializer`

`spring.kafka.producer.value-
serializer=org.springframework.kafka.support.serializer.JsonSerializer`

`spring.kafka.producer.properties.spring.json.add.type.headers=false`

Step 4 — Create the Order Event

Create:

Order Service

└─ src/main/java/com/example/orderservice/event
 └─ OrderCreatedEvent.java

Code:

```
package com.example.orderservice.event;
```

```
public class OrderCreatedEvent {
```

```
    private Long orderId;
```

```
    private Long productId;
```

```
    private Integer quantity;
```

```
    public OrderCreatedEvent() {
```

```
    }
```

```
    public OrderCreatedEvent(Long orderId, Long productId, Integer quantity) {
```

```
        this.orderId = orderId;
```

```
        this.productId = productId;
```

```
        this.quantity = quantity;
```

```
    }
```

```
    public Long getOrderId() {
```

```
        return orderId;
```

```
    }
```

```
    public void setOrderId(Long orderId) {
```

```
        this.orderId = orderId;
```

```
    }
```

```
public Long getProductId() {  
    return productId;  
}  
  
public void setProductId(Long productId) {  
    this.productId = productId;  
}  
  
public Integer getQuantity() {  
    return quantity;  
}  
  
public void setQuantity(Integer quantity) {  
    this.quantity = quantity;  
}  
}
```

Step 5 — Create Kafka Producer

Create:

```
Order Service  
└─ event  
    └─ OrderEventProducer.java
```

Code:

```

package com.example.orderservice.event;

import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderEventProducer {

    private static final String TOPIC = "order-created";

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventProducer(
        KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishOrderCreated(OrderCreatedEvent event) {

        kafkaTemplate.send(
            TOPIC,
            event.getOrderId().toString(),
            event
        );
    }
}

```

This publishes the event to:

order-created

Step 6 — Publish the Event When an Order Is Created

In your existing OrderService.java, inject:

```
private final OrderEventProducer orderEventProducer;
```

Constructor:

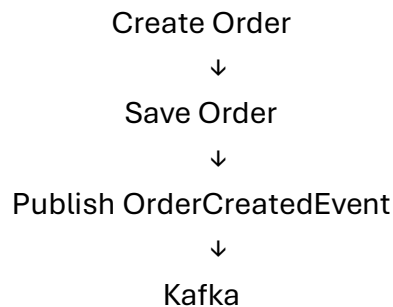
```
public OrderService(OrderEventProducer orderEventProducer) {  
    this.orderEventProducer = orderEventProducer;  
}
```

After successfully creating an order:

```
OrderCreatedEvent event = new OrderCreatedEvent(  
    order.getId(),  
    order.getProductId(),  
    order.getQuantity()  
);
```

```
orderEventProducer.publishOrderCreated(event);
```

So, the flow becomes:



Step 7 — Configure Notification Service

Open:

```
Notification Service  
└─ src/main/resources/application.properties
```

Add:

```
spring.kafka.bootstrap-servers=localhost:9092
```

```
spring.kafka.consumer.group-id=notification-service
```

```
spring.kafka.consumer.auto-offset-reset=earliest
```

```
spring.kafka.consumer.key-  
deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

```
spring.kafka.consumer.value-  
deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
```

```
spring.kafka.consumer.properties.spring.json.trusted.packages=com.example.*
```

Step 8 — Create Event Class in Notification Service

Create:

Notification Service

```
└─ src/main/java/com/example/notificationservice/event  
    └─ OrderCreatedEvent.java
```

Use the same fields:

```
package com.example.notificationservice.event;
```

```
public class OrderCreatedEvent {  
  
    private Long orderId;  
    private Long productId;  
    private Integer quantity;  
  
    public OrderCreatedEvent() {  
    }  
  
    public Long getOrderId() {  
        return orderId;  
    }  
  
    public void setOrderId(Long orderId) {  
        this.orderId = orderId;  
    }  
  
    public Long getProductId() {
```

```

        return productId;
    }

    public void setProductId(Long productId) {
        this.productId = productId;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }
}

```

Step 9 — Create Kafka Consumer

Create:

Notification Service

└─ event

└─ OrderEventConsumer.java

Code:

```
package com.example.notification.service.event;
```

```
import org.springframework.kafka.annotation.KafkaListener;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class OrderEventConsumer {
```

```
    @KafkaListener(
```

```
        topics = "order-created",
```

```
        groupId = "notification-service"
```

```
    )
```



```
public void consume(OrderCreatedEvent event) {  
  
    System.out.println(  
        "Order received: " + event.getOrderId()  
    );  
  
    System.out.println(  
        "Sending notification for order: "  
        + event.getOrderId()  
    );  
}  
}
```

Step 10 — Test the complete flow

Start:

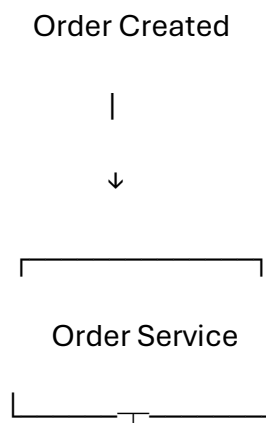
1. Kafka
2. Order Service
3. Notification Service

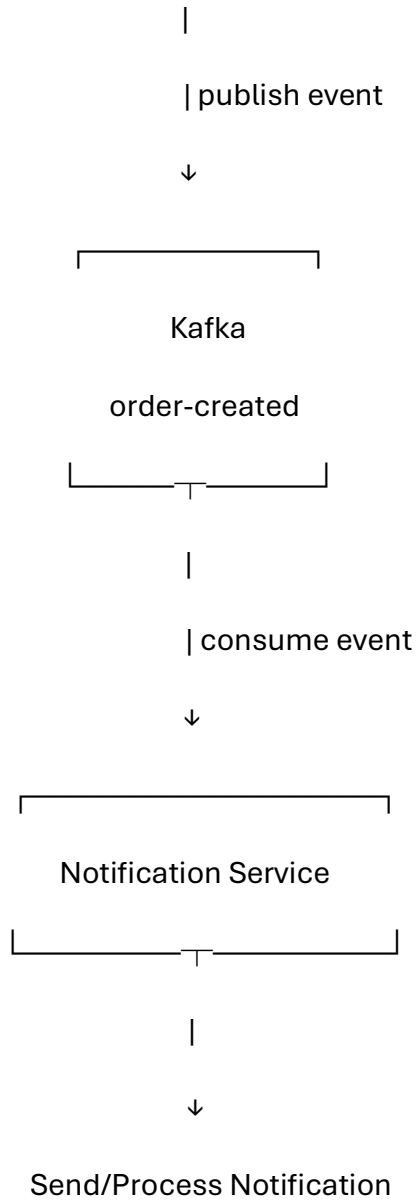
Create an order through your Order Service.

For example:

POST /orders

The flow should be:





You should see something similar in the **Notification Service console**:

Order received: 101

Sending notification for order: 101

Requirement

Order Service

Kafka topic

Event publishing

Completed



order-created



Notification Service



Event consumption



Kafka consumer group

notification-service

Task 04 is completed: Order Service publishes an event to Kafka, and

Notification Service consumes it.